

Jest

Jest es un *framework* de pruebas en JavaScript desarrollado inicialmente por Facebook. Su función principal es **ejecutar pruebas, verificar que los resultados sean los esperados y generar reportes**. No sabe nada de React específicamente; es un entorno para probar cualquier código JavaScript o TypeScript.

¿Para qué sirve?

- Ejecutar los archivos de prueba (`.test.js` o `.spec.js`).
- Verificar si un valor es igual a otro.
- Simular funciones o módulos externos.

Características principales:

- **Aserciones integradas:** Funciones como `expect(a).toBe(b)` para validar resultados.
- **Mocking potente:** Permite simular llamadas a APIs, módulos de terceros o funciones complejas para aislar el código que estás probando.
- **Velocidad (Ejecución en paralelo):** Ejecuta las pruebas en procesos paralelos, lo que lo hace muy rápido en proyectos grandes.
- **Reportes de Cobertura (Coverage):** Puede decirte exactamente qué porcentaje de tu código (líneas, funciones, ramas lógicas) está cubierto por las pruebas con solo añadir un *flag* (`--coverage`).
- **Pruebas de Instantáneas (Snapshot Testing):** Guarda una "foto" de la estructura de tu código (o componente) y te avisa si cambia en el futuro de forma inesperada.

React Testing Library

React Testing Library es una librería de utilidades de prueba específica para React, creada por Kent C. Dodds. Su función principal es **renderizar tus componentes en un entorno virtual y permitirte interactuar con ellos como lo haría un usuario real**.

¿Para qué sirve?

- Renderizar componentes de React en un DOM simulado (`jsdom`).
- Buscar elementos en la pantalla (botones, textos, imágenes).
- Simular eventos de usuario (hacer clic, escribir en un input, enviar un formulario).

Características principales:

- **Filosofía centrada en el usuario:** Su regla de oro es: *"Cuanto más se parezcan tus pruebas a la forma en que se usa tu software, más confianza te darán"*. No te permite buscar elementos por detalles técnicos (como el nombre de la clase CSS o el estado

interno de React), sino por cosas que un usuario ve (como un texto o el rol del elemento).

- **Fomento de la accesibilidad:** Te empuja a usar consultas como `getByRole` (ej. buscar un elemento con rol de "botón" y nombre "Enviar"), lo que indirectamente te obliga a escribir código HTML semántico y accesible.
- **Independencia del código interno:** Si refactorizas un componente (por ejemplo, pasas de usar Clases a Hooks) pero el componente visualmente sigue haciendo lo mismo, la prueba con RTL **no se romperá**. Solo le importa el resultado final, no cómo llegaste a él.

Comando para instalar Jest y RTL: `npm install --save-dev jest jest-environment-jsdom @testing-library/react @testing-library/jest-dom @testing-library/user-event`

para verificar que se instaló correctamente: `nano package.json` y en `devDependencies` deberían aparecer las librerías.

Desde Jest 28 el entorno que simula el navegador viene desactivado por defecto, crear un archivo llamado [jest.config.js](#) y ponerle el siguiente contenido: **export default {**

```
  testEnvironment: 'jest-environment-jsdom',  
  
};
```

Configurar el script de ejecución: abrir `package.json` y agregar: **"scripts": {**

```
  "test": "jest"  
  
} en la sección de scripts.
```

Para verificar ejecutar `npm test` y ver que sale error de no haber encontrado archivos de prueba.

probar código:

instalar babel (transpilador de JSX a JS) con : `npm install --save-dev babel-jest @babel/core @babel/preset-env @babel/preset-react`

crear un archivo al mismo nivel que `package.json` llamado **babel.config.cjs** con el siguiente código: **module.exports = {**

```
presets: [  
  ['@babel/preset-env', { targets: { node: 'current' } }],  
  ['@babel/preset-react', { runtime: 'automatic' }],  
],  
};
```